

Testing a User-Based To-Do List Application with Google Test (gtest)

Eazdan Mostafa Rafin

Roll: 2003055

June 15, 2025

1 Objective

The objective of this report is to document the unit testing of a simple user-based to-do list application. The goal is to verify that all core backend functionalities—user registration, login, logout, task addition, and task deletion—behave correctly. Using GoogleTest (gtest) as the testing framework, each feature is tested through automated test cases to detect any logic errors. The report outlines the testing strategy, shows the relevant C++ code (both application logic and test cases), and presents sample output to demonstrate the tests in action.

2 Introduction

Unit testing is a fundamental practice in software engineering to ensure code correctness and maintainability. In this project, we focus on the backend of a to-do list application that manages user accounts and task entries. The application provides functions for registering new users, authenticating logins, logging out, adding to-do tasks, and deleting existing tasks. To validate this functionality, we use GoogleTest, Google’s C++ testing framework. GoogleTest is based on the xUnit architecture and offers a suite of assertion macros and test runners that simplify writing and executing unit tests. By structuring tests into test suites and using clear assertions, GoogleTest helps isolate failures and ensures consistent verification of the application’s behavior across different environments.

3 Methodology

The testing methodology involves the following steps and considerations:

- **Test Environment:** We developed the application and tests in C++ and integrated GoogleTest for unit testing. All tests are run on the same platform to ensure repeatability.

- **Feature Identification:** Each core feature of the application is targeted by tests. Specifically, we wrote tests for user registration (including duplicate username handling), login (correct and incorrect credentials), logout behavior, adding tasks (when logged in and not logged in), and deleting tasks (valid and invalid indices).
- **Test Organization:** Tests are organized into logically related suites. For example, the `UserTests` suite covers registration and authentication, while the `TaskTests` suite covers task management. Grouping tests in this way reflects the structure of the code and makes the suite easy to maintain.
- **Assertions:** For each test case, we use GoogleTest assertions such as `EXPECT_TRUE`, `EXPECT_FALSE`, and `EXPECT_EQ` to verify expected outcomes. These assertions provide detailed feedback if a check fails, including the expected and actual values.
- **Test Execution:** The tests are compiled and executed together. GoogleTest runs each test independently, reporting the result of each. We capture the console output of this test run to include in this report.

4 Code

4.1 Application Code

The core application logic is implemented in C++ as shown below. A `ToDoList` class manages user accounts and tasks. Users are stored with their usernames, passwords, and a list of tasks. A logged-in user is tracked by an index. Methods enforce basic checks (e.g., unique usernames, valid login credentials, and valid task indices) and return boolean flags indicating success or failure of operations.

Listing 1: C++ application code for the to-do list logic.

```
#include <string>
#include <vector>
using namespace std;

class ToDoList {
public:
    struct User {
        string username;
        string password;
        vector<string> tasks;
    };

private:
    vector<User> users;
    int currentUserIndex;

public:
    ToDoList() : currentUserIndex(-1) {}
```

```

bool registerUser(const string& username, const string& password
) {
    for (const auto& user : users) {
        if (user.username == username) {
            return false; // username already exists
        }
    }
    users.push_back(User{username, password, {}});
    return true;
}

bool login(const string& username, const string& password) {
    for (int i = 0; i < (int)users.size(); ++i) {
        if (users[i].username == username && users[i].password
            == password) {
            currentUserIndex = i;
            return true;
        }
    }
    return false; // no matching user or wrong password
}

void logout() {
    currentUserIndex = -1;
}

bool addTask(const string& task) {
    if (currentUserIndex < 0 || currentUserIndex >= (int)users.
        size()) {
        return false; // no user logged in
    }
    users[currentUserIndex].tasks.push_back(task);
    return true;
}

bool deleteTask(int index) {
    if (currentUserIndex < 0 || currentUserIndex >= (int)users.
        size()) {
        return false; // no user logged in
    }
    auto& tasks = users[currentUserIndex].tasks;
    if (index < 0 || index >= (int)tasks.size()) {
        return false; // invalid index
    }
    tasks.erase(tasks.begin() + index);
    return true;
}

```

```

    }

    int getTaskCount() const {
        if (currentUserIndex < 0 || currentUserIndex >= (int)users.
            size()) {
            return 0;
        }
        return users[currentUserIndex].tasks.size();
    }
};

```

4.2 Test Cases (GoogleTest)

The following code shows GoogleTest unit tests for the above application logic. We include the GoogleTest header and our application header. The tests check both positive and negative cases. For example, registering a new user should succeed, while attempting to register a duplicate username should fail. Similarly, login/logout and task operations are verified. The `main` function initializes GoogleTest and runs all tests.

Listing 2: GoogleTest cases for the to-do list application.

```

#include <gtest/gtest.h>
#include "ToDoList.h"

// Test suite for user registration and login/logout
TEST(UserTests, RegisterSuccess) {
    ToDoList app;
    EXPECT_TRUE(app.registerUser("alice", "pw"));
}

TEST(UserTests, RegisterDuplicate) {
    ToDoList app;
    app.registerUser("alice", "pw");
    EXPECT_FALSE(app.registerUser("alice", "pw"));
}

TEST(UserTests, LoginSuccess) {
    ToDoList app;
    app.registerUser("bob", "pw2");
    EXPECT_TRUE(app.login("bob", "pw2"));
}

TEST(UserTests, LoginFailure) {
    ToDoList app;
    app.registerUser("bob", "pw2");
    EXPECT_FALSE(app.login("bob", "wrong"));
}

TEST(UserTests, LogoutWithoutLogin) {
    ToDoList app;
    app.logout();
    EXPECT_FALSE(app.addTask("task"));
}

```

```

}
TEST(UserTests, LogoutAfterLogin) {
    ToDoList app;
    app.registerUser("charlie", "pass3");
    app.login("charlie", "pass3");
    app.logout();
    EXPECT_FALSE(app.addTask("newtask"));
}

// Test suite for task addition and deletion
TEST(TaskTests, AddTaskLoggedIn) {
    ToDoList app;
    app.registerUser("d", "pw");
    app.login("d", "pw");
    EXPECT_TRUE(app.addTask("t1"));
    EXPECT_EQ(app.getTaskCount(), 1);
}
TEST(TaskTests, AddTaskNotLoggedIn) {
    ToDoList app;
    EXPECT_FALSE(app.addTask("t1"));
    EXPECT_EQ(app.getTaskCount(), 0);
}
TEST(TaskTests, DeleteTaskLoggedIn) {
    ToDoList app;
    app.registerUser("e", "pw");
    app.login("e", "pw");
    app.addTask("t1");
    app.addTask("t2");
    EXPECT_EQ(app.getTaskCount(), 2);
    EXPECT_TRUE(app.deleteTask(0));
    EXPECT_EQ(app.getTaskCount(), 1);
    EXPECT_FALSE(app.deleteTask(5));
}
TEST(TaskTests, DeleteTaskNotLoggedIn) {
    ToDoList app;
    EXPECT_FALSE(app.deleteTask(0));
}

int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}

```

5 Output

Below is a sample console output from running the above test suite. Each test is reported as either OK or FAILED. In this sample run, all tests pass. This output demonstrates that the expected behaviors (both success and failure cases) are correctly handled.

```
[=====] Running 10 tests from 2 test suites.
[-----] Global test environment set-up.
[-----] 6 tests from UserTests
[ RUN      ] UserTests.RegisterSuccess
[          OK ] UserTests.RegisterSuccess (0 ms)
[ RUN      ] UserTests.RegisterDuplicate
[          OK ] UserTests.RegisterDuplicate (0 ms)
[ RUN      ] UserTests.LoginSuccess
[          OK ] UserTests.LoginSuccess (0 ms)
[ RUN      ] UserTests.LoginFailure
[          OK ] UserTests.LoginFailure (0 ms)
[ RUN      ] UserTests.LogoutWithoutLogin
[          OK ] UserTests.LogoutWithoutLogin (0 ms)
[ RUN      ] UserTests.LogoutAfterLogin
[          OK ] UserTests.LogoutAfterLogin (0 ms)
[-----] 6 tests from UserTests (1 ms total)
[-----] 4 tests from TaskTests
[ RUN      ] TaskTests.AddTaskLoggedIn
[          OK ] TaskTests.AddTaskLoggedIn (0 ms)
[ RUN      ] TaskTests.AddTaskNotLoggedIn
[          OK ] TaskTests.AddTaskNotLoggedIn (0 ms)
[ RUN      ] TaskTests.DeleteTaskLoggedIn
[          OK ] TaskTests.DeleteTaskLoggedIn (0 ms)
[ RUN      ] TaskTests.DeleteTaskNotLoggedIn
[          OK ] TaskTests.DeleteTaskNotLoggedIn (0 ms)
[-----] 4 tests from TaskTests (1 ms total)
[-----] Global test environment tear-down
[=====] 10 tests from 2 test suites ran. (2 ms total)
[ PASSED  ] 10 tests.
```

6 Discussion

The test suite execution shows that all 10 test cases passed, indicating that the to-do list application's core features work as expected under tested conditions. The tests covered both normal and edge cases (such as duplicate registration or invalid task indices), providing confidence in the correctness of the logic. By running each test independently, GoogleTest clearly reports which test succeeded or failed, allowing quick pinpointing of issues if any had occurred. In this experiment, no failures occurred, so no logical defects were uncovered. However, this testing is limited to basic backend functionality; in a production scenario, one

might add more tests for robustness (e.g., boundary cases, concurrent access, or additional features).

GoogleTest's framework made it straightforward to write these tests in a structured way. For example, grouping tests into suites (`UserTests` and `TaskTests`) reflects the application's modular design and aids maintainability. The use of descriptive assertions (`EXPECT_TRUE`, `EXPECT_EQ`) ensures that any deviation would be immediately visible with expected vs actual values. Overall, the automated tests serve as a regression suite: future code changes can be verified quickly by rerunning this test suite.

7 Conclusion

In conclusion, we successfully applied unit testing to the to-do list application using GoogleTest. The automated test cases verified all specified functionalities of user management and task handling. The passing test results indicate that, for the tested scenarios, the application meets its design requirements. Incorporating GoogleTest into the development process helps improve code reliability by catching errors early. For future work, the testing framework can be expanded with more comprehensive cases (such as testing invalid inputs or integrating with a user interface layer) to further ensure quality. This exercise demonstrates how a simple yet systematic approach to testing—writing targeted test cases and validating output—can significantly enhance confidence in software correctness.